



A.D. 1308
unipg
DIPARTIMENTO
DI INGEGNERIA

Tesina Finale di
Programmazione di Interfacce Grafiche e Dispositivi Mobili
Corso di Laurea in Ingegneria Informatica ed Elettronica – A.A. 2021-2022
DIPARTIMENTO DI INGEGNERIA

docente
Prof. Luca GRILLI

PENALTY LEAGUE



applicazione desktop JFC/SWING

studente

329683 **Michele Mariotti** michele.mariotti@studenti.unipg.it

Data ultimo aggiornamento: 4 febbraio 2023

0. Indice

1	Descrizione del Problema	2
1.1	I calci di rigore (penalty shootout)	2
1.2	L'applicazione Penalty League	3
2	Specifica dei Requisiti	4
3	Progetto	5
3.1	Architettura del Sistema Software	5
3.2	Logic	6
3.3	View	10
3.4	Problemi Riscontrati	16
4	Possibili sviluppi futuri	18

1. Descrizione del Problema

L'obiettivo di questo lavoro è lo sviluppo di un'applicazione desktop, denominata *Penalty League*, che realizza una simulazione di calci di rigore.

L'applicazione sarà implementata utilizzando la tecnologia JFC/Swing in modo da favorire un'ampia portabilità su diversi sistemi operativi (piattaforme), riducendo al minimo eventuali modifiche al codice sorgente. Tuttavia, il codice prodotto sarà testato e ottimizzato per il sistema operativo Windows 11.

Di seguito sarà data una breve descrizione dei calci di rigore nella realtà, dopodiché si fornirà una descrizione del videogioco che si intende realizzare.

1.1 I calci di rigore (penalty shootout)

I calci di rigore sono un metodo per determinare il vincitore di una partita di calcio in caso di parità alla fine solitamente dei tempi supplementari. Sono utilizzati in competizioni ad eliminazione diretta, nei quali una delle due squadre deve necessariamente prevalere sull'altra per superare il turno.

Le squadre si alternano nella battuta dei rigori finché entrambe non ne hanno battuti cinque a testa; tuttavia se una squadra ha segnato più rigori dell'altra e questa nemmeno segnando tutti i rimanenti può eguagliarla, i calci di rigore terminano in anticipo.

Se alla fine dei dieci rigori totali le squadre hanno realizzato lo stesso numero di reti si continuerà “a oltranza” con un tiro per parte finché, dopo aver eseguito lo stesso numero di tiri, una squadra si trovi in vantaggio rispetto all'altra.

1.2 L'applicazione Penalty League

Il videogioco realizza una simulazione di un torneo ad eliminazione diretta di calci di rigore tra varie squadre. Dopo aver selezionato la propria squadra, viene sorteggiato in modo pseudo-randomico il tabellone del torneo. In ogni singola partita si alternano, come nella realtà, un turno di tiro ed un turno di parata:

- **Turno di tiro**

Questo turno adopera una meccanica di gioco basata sul tempismo in cui si clicca il tasto sinistro del mouse e si compone in tre fasi che si susseguono:

- *fase di selezione della direzione del tiro*: attraverso una linea verticale che si sposta in orizzontale, si seleziona la direzione desiderata cliccando nel momento in cui la linea si trova in tal posizione.
- *fase di selezione dell'altezza del tiro*: un indicatore si muove lungo l'asse verticale precedentemente selezionato e in modo totalmente analogo alla fase antecedente si deve cliccare nel momento opportuno.
- *fase di selezione della potenza del tiro*: all'interno di una barra oscilla un indicatore, si deve cliccare nell'istante giusto per selezionare una potenza il più elevata possibile.

La probabilità che il portiere avversario pari il tiro è condizionata dalla direzione, altezza e potenza del tiro.

- **Turno di parata**

Questo turno di gioco utilizza una meccanica basata sulla reattività e precisione: un indicatore mostra qualche attimo prima del calcio del pallone la posizione dove arriverà e per pararla si deve cliccare attraverso il mouse l'indicatore nel periodo di tempo in cui esso viene mostrato.

Se si vince la partita si passa a quella successiva in base al tabellone del torneo, altrimenti si perde di conseguenza il torneo.

Ad ogni squadra è associata una valutazione che determina la difficoltà del gioco in entrambi i turni: la velocità con cui si spostano gli indicatori nel turno di tiro e la durata del periodo in cui l'indicatore viene mostrato e deve essere cliccato nel turno di parata sono dipendenti dalla valutazione della squadra avversaria.

2. Specifica dei Requisiti

L'applicazione *Penalty League* soddisfa i seguenti requisiti:

1. Applicazione eseguita in un'unica finestra grafica.
2. L'interazione con l'utente si svolge per via grafica, privilegiando l'utilizzo del solo mouse.
3. Utilizzo dell'architettura *Logic - View*.
4. Presenza di un sottofondo sonoro disattivabile dalla schermata iniziale dell'applicazione.
5. Presenza di effetti sonori differenti in caso di gol o parata, disattivabili dalla schermata iniziale dell'applicazione.
6. Presenza di una schermata in cui è possibile selezionare la propria squadra tra le varie disponibili indicate con relativi stemmi.
7. Presenza di una schermata in cui viene mostrato il tabellone (successivamente aggiornato tra una partita e l'altra).
8. Animazioni del portiere e del tiratore.
9. Movimento fluido del pallone.
10. Mostrare tiratore con maglietta differente in base al team di appartenenza.
11. Presenza di un tabellino, aggiornato successivamente ad ogni rigore, che mostra il punteggio della partita.
12. Esibizione di una scritta in caso di gol/parata.
13. Livello di difficoltà determinato dalla valutazione della squadra avversaria.

3. Progetto

Viene ora descritta la struttura dell'applicazione realizzata, illustrandone prima l'architettura software per poi scendere nel dettaglio dei blocchi funzionali che la compongono.

3.1 Architettura del Sistema Software

Per la realizzazione del progetto si è scelto di basarsi sul pattern di programmazione *Logic - View*.

Il pacchetto Logic si occupa di gestire la logica degli aspetti generali del videogioco, come ad esempio la gestione della partita e del torneo; inoltre svolge il compito di stabilire il risultato delle interazioni dell'utente raccolte tramite il View e di gestire il comportamento dell'avversario.

Il pacchetto View si occupa, come detto in precedenza, di ricevere gli input dell'utente, del rendering grafico e dell'audio dell'applicazione.

I due pacchetti comunicano attraverso le due interfacce ILogic ed IView.

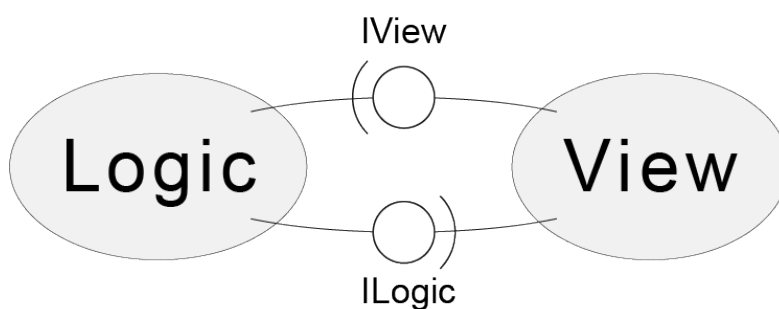


Figura 3.1: Architettura *Logic - View* del progetto, ove sono evidenziate le interfacce esposte dai singoli moduli architetturali al fine di ridurre l'accoppiamento tra classi di moduli distinti.

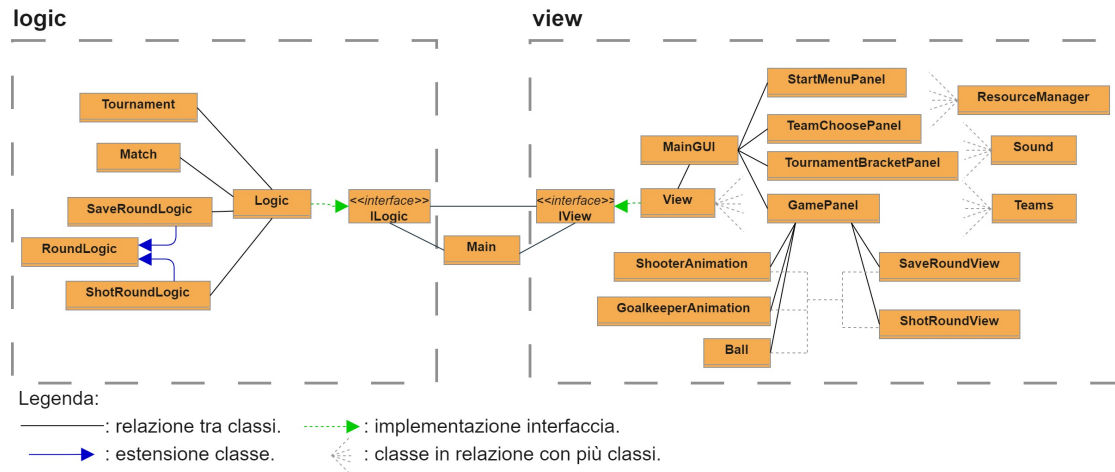


Figura 3.2: Diagramma che illustra le classi del progetto e le principali relazioni tra loro.

3.2 Logic

Le classi appartenenti al blocco Logic si trovano nel package `penaltyleague.logic`. La loro struttura è rappresentata nel diagramma UML in figura 3.3.

- *Tournament*

Questa classe si occupa della generazione del tabellone del torneo e della sua gestione.

- *Match*

Coordina la singola partita giocata dall'utente all'interno del torneo, gestendo il susseguirsi dei round di parata e tiro, il punteggio della partita ed attraverso quest'ultimo controlla se l'utente o il suo avversario hanno vinto la partita tramite i metodi `checkUserWin()` e `checkOpponentWin()`.

- *RoundLogic*

Questa classe astratta contiene metodi che vengono utilizzati sia per la logica del turno di tiro che per quello di parata. Tra i metodi presenti abbiamo `isShotTowardGoal()`, che restituisce `true` se il tiro è verso la porta o `false` in caso contrario, e `getRegion()` che date le coordinate del tiro o del tentativo di parata restituisce la regione della porta corrispondente (la porta è divisa in 6 regioni della stessa grandezza, 3 nella parte alta della porta ed altrettante nella bassa). Inoltre sono presenti i tre metodi `checkHitLeftPostAndGoal()`,

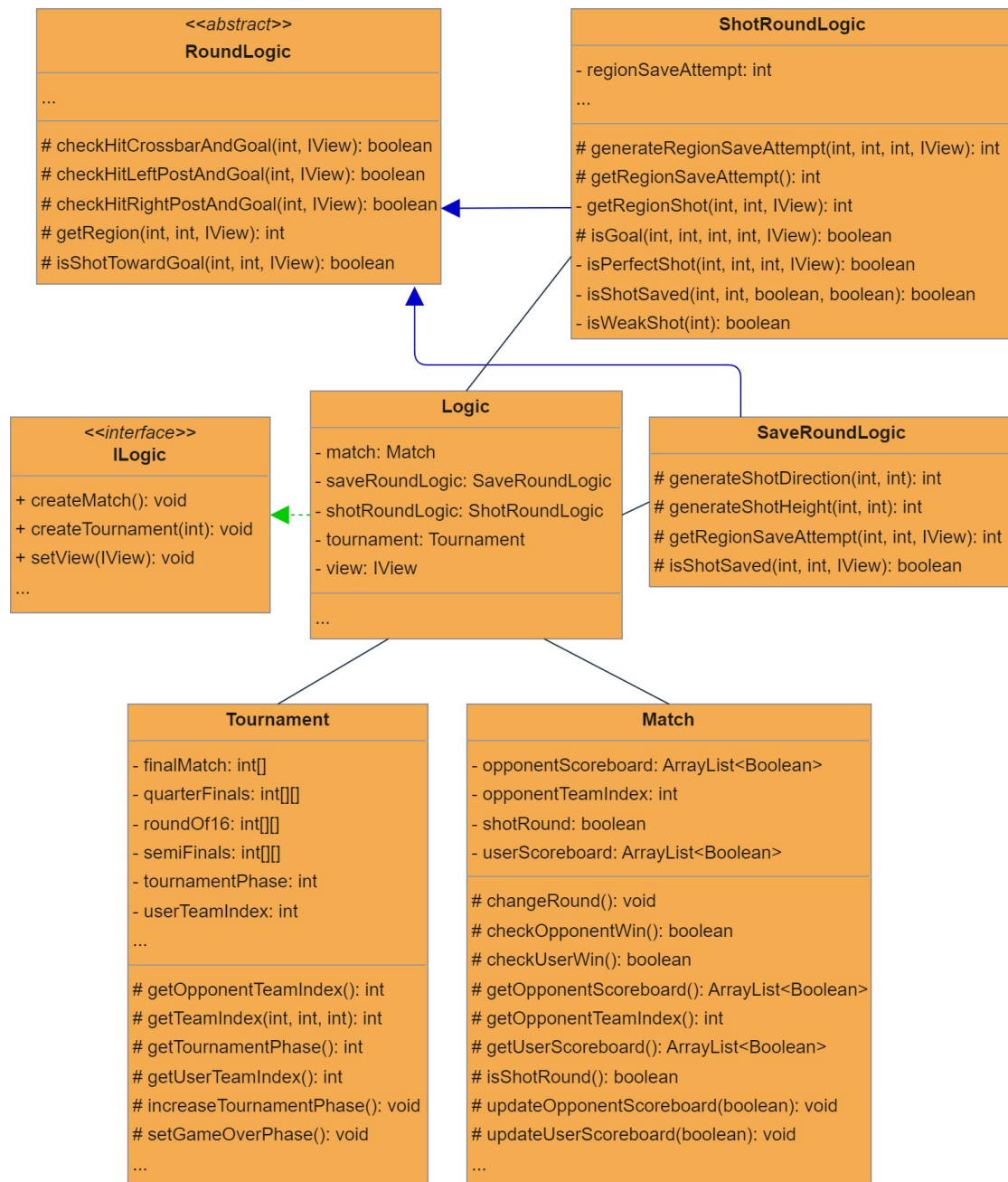


Figura 3.3: Diagramma UML delle classi di penaltyleague.logic

checkHitRightPostAndGoal() e *checkHitCrossbarAndGoal()* che data la direzione/altezza del tiro restituiscono *true* se il pallone colpisce il relativo palo e poi entra in porta (se non parato successivamente dal portiere), ovvero se viene colpita la metà interna del palo. Si è scelto di rendere questa classe astratta in quanto priva di implementazione se non estesa.

- *ShotRoundLogic*

Realizza la logica del turno di gioco in cui spetta all'utente tirare. Tra i suoi metodi troviamo:

- *generateRegionSaveAttempt()*: genera in modo pseudo-randomico la regione della porta dove il portiere avversario proverà a parare il tiro. Se il tiro è “debole” (vedi metodo *isWeakShot()*) la regione generata sarà la stessa del tiro.
- *isPerfectShot()*: restituisce *true* se il tiro è effettuato molto vicino ad uno dei quattro angoli della porta con una potenza elevata, rendendo il tiro impossibile da parare per il portiere avversario.
- *isWeakShot()*: restituisce *true* se il tiro è molto debole, rendendolo sempre parabile dal portiere, a prescindere dalla direzione ed altezza del tiro.
- *isShotSaved()*: avvalendosi dei tre metodi precedentemente illustrati, restituisce *true* se il tiro viene parato direttamente dal portiere avversario. Il tiro viene parato se la regione dove avviene il tentativo di parata corrisponde alla regione del tiro e non si tratta di un tiro “perfetto” (*isPerfectShot()* restituisce *false*), oppure se si tratta di un tiro “debole” (*isWeakShot()* restituisce *true*).
- *isGoal()*: restituisce *true* se il tiro è direzionato verso la porta e non viene parato dal portiere.

- *SaveRoundLogic*

Realizza la logica del turno di gioco in cui spetta all'utente parare. Tra i suoi metodi troviamo:

- *generateShotDirection()*: genera in modo pseudo-randomico la direzione del tiro effettuato dall'avversario.
- *generateShotHeight()*: genera in modo pseudo-randomico l'altezza del tiro effettuato dall'avversario.
- *isShotSaved()*: restituisce *true* se il tiro viene parato dall'utente. Questo metodo viene utilizzato per verificare se l'utente clicca nell'indicatore o

al di fuori, solo nel primo caso parerà il tiro (questo passaggio sarà più chiaro dopo aver letto il capitolo riguardante il blocco View).

- *Logic e ILogic* (fig. 3.4)

La classe Logic implementa l'interfaccia ILogic, che include tutti i metodi necessari per la comunicazione tra i due package logic e view. Essa contiene le istanze delle classi ShotRoundLogic, SaveRoundLogic, Tournament e Match; attraverso queste implementa i vari metodi dell'interfaccia. Ad ogni nuovo torneo viene creato un nuovo oggetto Tournament che viene salvato nella variabile di istanza tournament, andando a prendere il posto del precedente oggetto. Lo stesso discorso vale per la variabile di istanza match, che viene aggiornata ad ogni partita. Inoltre abbiamo un oggetto IView, che permette di utilizzare i metodi dell'interfaccia per comunicare con il pacchetto view.

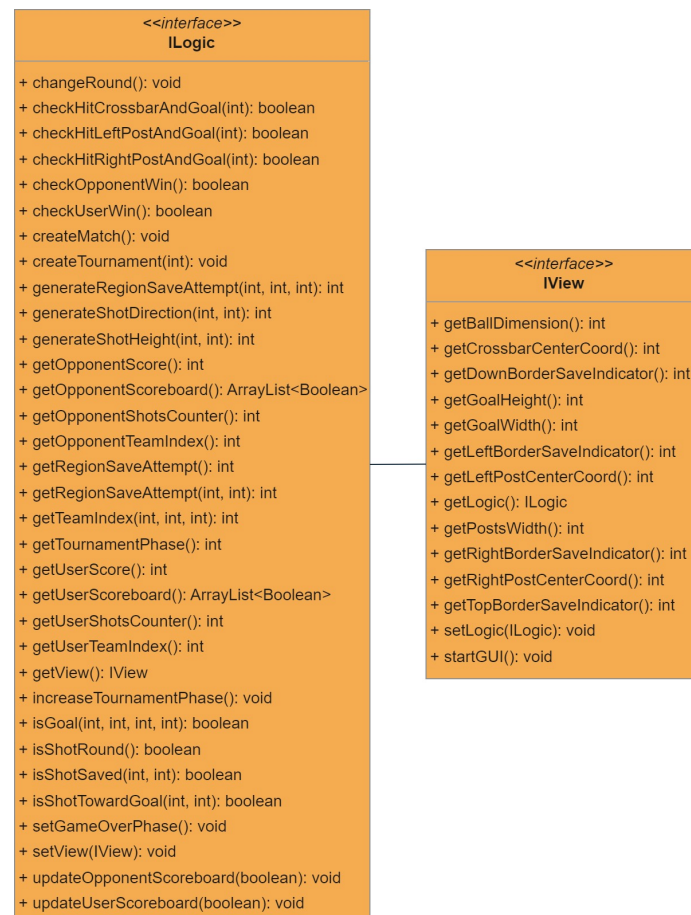


Figura 3.4: Diagramma UML delle interfacce ILogic e IView.

3.3 View

Le classi appartenenti al blocco View si trovano nel package `penaltyleague.view`. La loro struttura è rappresentata nei diagrammi UML nelle figure 3.5 e 3.6.

- *MainGUI*

Estende `JFrame` e costituisce la finestra dell'applicazione. La finestra ha come dimensione la larghezza dello schermo dove viene eseguito il videogioco e la corrispondente altezza che mantiene il formato 16:9. Al suo interno sono definiti i metodi che permettono di cambiare il pannello mostrato all'interno della finestra.

- *ResourceManager*

Offre dei metodi statici che permettono di caricare le risorse necessarie.

- *StartMenuPanel*

Estende `JPanel` ed è il pannello iniziale del videogioco. Da qui si può scegliere se permettere la riproduzione di suoni oppure mutarli.

- *TeamChoosePanel*

Estende `JPanel` ed è il pannello che permette all'utente di scegliere la propria squadra cliccando nella relativa `JLabel`. Una volta scelta la squadra viene invocato il metodo `createTournament()` dell'interfaccia `ILogic` passando come parametro la squadra scelta (in particolare il suo indice, vedi classe `Teams`) e si passa al pannello `TournamentBracketPanel`.

- *TournamentBracketPanel*

Estende `JPanel` ed è il pannello in cui viene mostrato il tabellone del torneo in base all'avanzamento dell'utente nel torneo stesso. A fianco agli stemmi delle squadre è indicato in stelle il rispettivo livello, ad eccezione della squadra dell'utente, in cui viene scritto "your team" in quanto il livello della squadra scelta non incide nella difficoltà del gioco. Questo pannello viene utilizzato anche per mostrare la schermata di vittoria o sconfitta, ovvero quando si vince la finale oppure si perde una qualsiasi altra partita e quindi si esce di conseguenza dal torneo.

- *Sound*

Questa classe contiene tutti i suoni che vengono riprodotti all'interno dell'applicazione ed offre i relativi metodi per azionarli/disattivarli.

- *Teams*

Questa classe contiene tutti gli aspetti riguardanti le varie squadre, come i nomi, il loro rating, i loghi e le immagini dei rispettivi tiratori. L'indice della squadra all'interno di ogni array non varia e viene utilizzato per accedere al relativo elemento all'interno dell'array, per questo nel pacchetto logic si fa riferimento sempre solamente all'indice dei team.

- *GamePanel*

Estende JPanel ed è il pannello in cui si svolge il videogioco vero e proprio. Essendo la schermata fondamentale e più complessa dell'applicazione, è stata scomposta in più classi che realizzano ognuna un aspetto del gioco che viene mostrato nel pannello stesso: le classi *ShotRoundView*, *SaveRoundView*, *Ball*, *GoalkeeperAnimation* e *ShooterAnimation*. Tra i task di cui si occupa direttamente questa classe abbiamo:

- il rilevamento degli input del mouse, implementando l'interfaccia *MouseListener* (tramite il metodo *mousePressed()*, scomposto in *handleMousePressedSaveRound()* e *handleMousePressedShotRound()*)
- restituire le coordinate e dimensioni di elementi all'interno della schermata, come le coordinate dei pali o la grandezza della porta
- effettuare il movimento della rete tramite metodi richiamati in caso di gol (*moveNet()* per muoverla e *resetNet()* per resettarla alla posizione iniziale a fine turno)
- disegnare la schermata ed i suoi elementi tramite il metodo *paintComponent()* che ad ogni repaint richiama a sua volta vari metodi tra i quali *drawScoreboards()*, *drawShotRound()*, *drawSaveRound()* ed altri di tipo “draw” presenti nelle classi associate al GamePanel.

- *ShotRoundView*

Questa classe si occupa del turno di tiro dell'utente, in particolare della selezione della direzione, altezza e potenza del tiro (vedi descrizione nel paragrafo 1.2). Il metodo *drawSelectors()* disegna i selettori che vengono spostati dall'*actionPerformed()* ad ogni evento del timer. La selezione avviene tramite i metodi *setDirection()*, *setHeight()* e *setPower()*, invocati dal GamePanel alla pressione del mouse. Inoltre gestisce la fine del turno: invoca i metodi del Logic per stabilire se si segna, riproduce i vari suoni, aggiorna il tabellino, muove la rete richiamando il metodo *moveNet()* di GamePanel in caso di gol ed effettua il reset dei vari componenti ed il cambio di turno, controllando se una delle due squadre ha vinto la partita.

- *SaveRoundView*

Questa classe si occupa del turno di parata dell'utente, in particolare della selezione delle coordinate del tentativo di parata (vedi descrizione nel paragrafo 1.2). Il metodo *drawIndicator()* disegna l'indicatore che viene reso visibile o meno invocando il metodo *setIndicatorVisible()*. La selezione avviene tramite il metodo *setCoordSaveAttempt()* invocato dal *GamePanel* quando viene premuto il mouse mentre è visibile l'indicatore. Come in *ShotRoundView*, anche in questo caso la classe gestisce la fine del turno.

- *Ball*

Si occupa del pallone e del suo movimento:

- il metodo *moveTo()* avvia il timer e calcola la durata del tiro che dipende dalla sua potenza.
- l'*actionPerformed()* del timer sposta il pallone seguendo la retta che congiunge il dischetto del rigore dove è inizialmente posizionato e le coordinate finali del tiro (l'entità dello spostamento varia in base alla durata del tiro). In aggiunta alterna i due frame del pallone imitando il suo girare.
- una volta che il pallone arriva a destinazione, si controlla se viene colpito un palo interno e viene fatto cadere simulando la gravità

- *ShooterAnimation*

Realizza l'animazione del tiratore ed i metodi necessari per disegnarlo: il metodo *drawCurrentFrame()* si occupa di disegnare il frame corrente identificato da un indice che viene aumentato dall'*actionPerformed()* del timer. Una volta arrivato all'ultimo frame in cui il tiratore calcia il pallone, invoca i metodi *start()* di *GoalkeeperAnimation* e *moveTo()* di *Ball*, attivando i rispettivi timer. Nel turno di parata è il primo timer ad essere attivato ed invoca i metodi necessari per generare le coordinate del tiro e rendere visibile l'indicatore.

- *GoalkeeperAnimation*

In modo analogo a quanto detto in *ShooterAnimation*, realizza l'animazione del portiere ed i metodi necessari per disegnarlo. In questo caso è necessario settare le coordinate dove riprodurre l'immagine in base a dove ha luogo il tentativo di parata: ciò avviene tramite i tre metodi *setLeftDivePosition()*, *setCenterDivePosition()* e *setRightDivePosition()*.

- *View e IView* (fig. 3.4)

La classe *View* implementa l'interfaccia *IView*. I metodi definiti nell'interfaccia *IView* ed implementati nel *View* restituiscono coordinate e dimensioni di elementi fondamentali all'interno della schermata di gioco (come i pali o la grandezza della porta). Essi sono stati necessari per la realizzazione di vari metodi nel pacchetto *logic*. Viceversa abbiamo un oggetto *ILogic* utilizzato per il medesimo scopo ed accessibile tramite il metodo *getLogic()*. Oltre a contenere i metodi dell'interfaccia, nella classe *View* sono presenti metodi utili alle varie classi del pacchetto, come *getUnitOfMeasureScreen()* che restituisce la larghezza della finestra diviso 1920: moltiplicando la dimensione dei vari elementi rappresentati all'interno della finestra per il valore restituito da questo metodo, essi avranno la giusta dimensione a prescindere dalla dimensione dello schermo (quanto detto vale anche per l'entità degli spostamenti e le coordinate degli elementi). Un altro compito che svolge la classe *View* è istanziare e rendere accessibili la *MainGUI*, i 4 pannelli e le classi *Sound* e *Teams*.

Main

Al di fuori di entrambi i pacchetti è presente la classe *Main* che permette l'esecuzione del programma, istanziando il *Logic* ed il *View* e collegandoli l'uno all'altro; successivamente invocando il metodo *startGUI()* di *View* crea e rende visibile la finestra, aggiungendo il pannello iniziale.

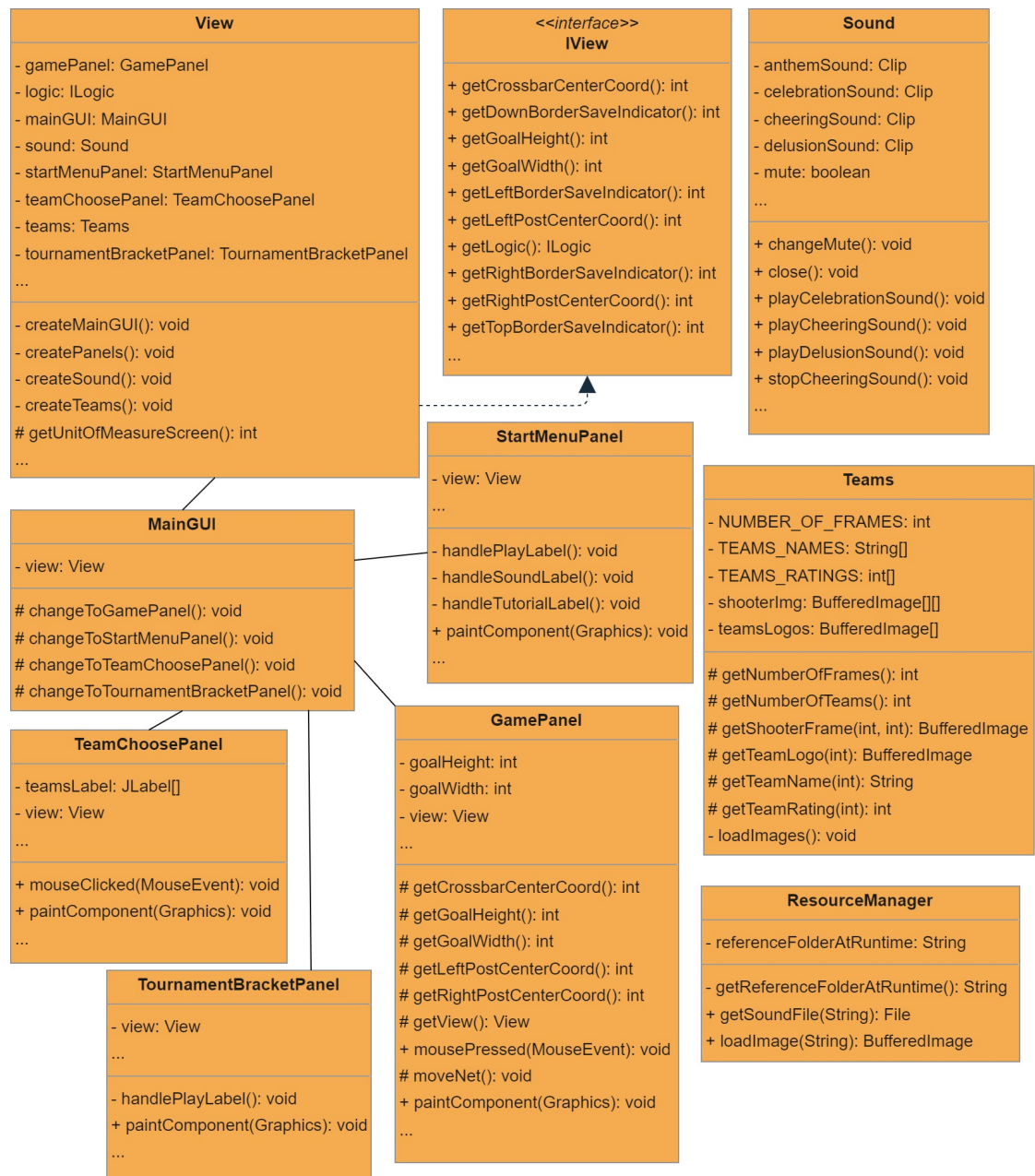


Figura 3.5: Diagramma UML delle classi di penaltyleague.view (escluse le classi riguardanti la schermata di gioco)



Figura 3.6: Diagramma UML delle classi di penaltyleague.view riguardanti la schermata di gioco

3.4 Problemi Riscontrati

Della parte logic dell'applicazione il vero ostacolo è stato la fase di progettazione della stessa. In particolare è stato difficile decidere l'idea di base che regola il comportamento del tiratore e del portiere dell'avversario dell'utente. Una volta aver progettato come farla, la realizzazione non ha comportato grosse difficoltà.

Per quanto riguarda la parte view, le problematiche sono state molto più numerose ed hanno richiesto una considerevole mole di lavoro. Di seguito vengono riportate le principali o più "particolari":

- rendere movimento selettori turno di tiro più fluido possibile: sono state provate numerose combinazioni tra delay timer ed entità dello spostamento dei selettori fino a trovare il più adeguato, tenendo conto della variazione in base alla difficoltà.
- realizzare e coordinare le varie animazioni.
- renderizzare comportamento del portiere avversario: ad esempio se si effettua un tiro "debole" al di fuori dello specchio della porta, il portiere non dovrà essere "forzato" a quella posizione.
- trovare il giusto compromesso sulla difficoltà del gameplay nei turni del gioco: ho deciso di rendere il turno di tiro più facile rispetto al turno di parata, come nella realtà è statisticamente più facile segnare un rigore piuttosto che pararlo.
- la realizzazione di un'applicazione la cui finestra si adatta alla larghezza dello schermo del computer su cui viene eseguita ha comportato una adeguata gestione delle dimensioni, coordinate e spostamenti dei vari elementi grafici.
- inizialmente nel GamePanel era stato implementato il metodo *mouseClicked()* dell'interfaccia *MouseListener*, ma provando il gioco si notava che alcune volte l'input dell'utente non veniva raccolto. Il motivo risiedeva nel fatto che se tra la pressione del mouse ed il rilascio avveniva un movimento del puntatore, allora l'input non veniva rilevato. Per questo si è spostata la gestione dell'input dell'utente nel metodo *mousePressed()*, che viene richiamato alla pressione del mouse.
- nei sistemi Windows è presente una impostazione dello schermo chiamata *ridimensionamento* che modifica la dimensione di testo, app ed altri elementi in modo tale da migliorare l'esperienza degli utenti che utilizzano pc con schermi di piccole dimensioni ma ad elevata risoluzione. Questa impostazione se non impostata a 100% comportava l'errato rilevamento della risoluzione

dello schermo (ad esempio uno schermo con risoluzione 1920x1080 se aveva l'impostazione a 125% veniva considerato come 1536x864) e gli oggetti `ImageIcon` venivano renderizzati in modo sfocato e con dimensioni errate. Si è risolto questo problema chiamando il seguente metodo all'interno del main: *`System.setProperty("sun.java2d.uiScale", "1.0")`*.

Un altro aspetto problematico è stato la fase di progettazione dell'applicazione nel suo complesso. Nel momento di realizzazione sono state fatte numerose modifiche rispetto all'idea iniziale, che hanno comportato a sua volta altre variazioni sulle parti già realizzate.

4. Possibili sviluppi futuri

La struttura del videogioco permette di modificare le squadre presenti andando a variare nel codice solamente i nomi delle squadre e le relative valutazioni all'interno della classe `Teams`, permettendo di creare molto agevolmente ad esempio una versione del gioco dedicata alle nazionali oppure un aggiornamento in base alla stagione agonistica.

Una possibile miglioria potrebbe essere l'utilizzo delle immagini in formato `svg` tramite la libreria esterna *Apache Batik*, garantendo una migliore resa grafica a prescindere dalle dimensioni dello schermo, in quanto gran parte delle immagini sono state realizzate in grafica vettoriale e successivamente esportate in `png`.